

rtl-mosaic: A Reuse-Aware Benchmark for LLM-Driven RTL Design

Leonardo Ferreira, Matthew Kotzbauer, Warren Zhu

Team 11, COMPSCI 1440R, Harvard University

{leonardo.ferreira@childrens.harvard.edu, matthewkotzbauer@college.harvard.edu, wzhu@college.harvard.edu}

Abstract—Existing LLM-for-Verilog benchmarks score models on scratch generation: given a spec, write the code. Real chip design is compositional; modern SoCs are 60–70% licensed IP by area. We built rtl-mosaic, a system-level harness that decomposes a Verilog spec into subblocks, routes reusable pieces to a curated 30-IP corpus over MCP, and stitches the result through Icarus Verilog testbenches. We then ran 15 frontier LLMs from four providers through the harness on 16 ChipBench problems, scoring each against hand-labeled gold. The headline finding: IP-reuse skill is its own axis. Routing F1 spreads from 0.13 to 0.52 across models while scratch pass rate on the same problems is roughly flat. Some of the strongest scratch coders are the weakest IP routers. A reuse-aware metric reveals a model capability that scratch-only benchmarks hide.

Index Terms—LLM, Verilog, RTL, IP reuse, benchmark, MCP, hardware design

I. INTRODUCTION

A modern SoC is 60–70% licensed IP by area [1]. Companies like Synopsys, Cadence, and Faraday ship thousands of pre-verified IP blocks, and re-authoring every primitive from scratch is incompatible with tape-out economics. Yet the dominant benchmarks for LLM-driven Verilog — VerilogEval [4], RTLLM, ChipBench [3] — score models on *scratch generation*: given a spec, write the code. None reward the agent for recognizing that an off-the-shelf register file would do. None penalize re-authoring it.

We argue this is the wrong incentive. The interesting question is not “can the model write a 200-line FIFO from gates” but “does the model recognize the FIFO is on the shelf and use it correctly?” To make this concrete we built rtl-mosaic — a system-level harness on top of SiliconMind-V1 [2] — and used it as the substrate for a multi-LLM benchmark.

Contributions. We introduce a reuse-aware evaluation protocol for Verilog generation. We ship a 30-IP corpus served over MCP, 16 hand-labeled gold-standard decomposition problems on top of ChipBench, and a deterministic keyword-overlap router. We run 15 frontier LLMs spanning Anthropic, OpenAI, AWS Bedrock (DeepSeek), and Google through the protocol — 240 evaluation calls. The result, summarized in Fig. 1, separates routing skill from scratch skill: F1 spread of 0.39 across models against a much narrower scratch-pass-rate spread on the same problems. We open-source the corpus, gold labels, and runner at <https://github.com/MattKotzbauer/rtl-mosaic>.

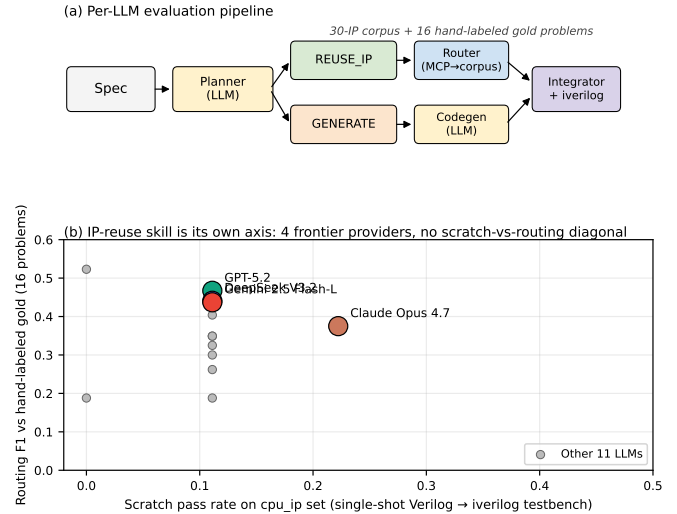


Fig. 1. (a) The rtl-mosaic evaluation pipeline. A spec goes to an LLM-as-planner that emits subblocks tagged REUSE_IP or GENERATE. Reusable subblocks resolve through a deterministic router over a 30-IP corpus served via MCP. Custom logic goes through codegen. The integrator stitches everything into a TopModule verified by Icarus Verilog. (b) The headline finding. Each dot is one of 15 frontier LLMs we evaluated. The four colored dots are best-of-provider models. Routing F1 (y-axis, vs. hand-labeled gold) and scratch Verilog pass rate on the same RISC-V CPU IP problems (x-axis) are not correlated. Strong scratch coders are not necessarily strong IP routers, and vice versa.

II. APPROACH

A. The pipeline

For each (LLM, problem) pair, we run the four-stage pipeline shown in Fig. 1(a). The *planner* receives the ChipBench spec and a fixed system prompt instructing it to return JSON: a list of one to six subblocks, each tagged REUSE_IP or GENERATE, with a port specification and a free-text search_query. The *router* is deterministic. For every REUSE_IP subblock it queries our 30-IP corpus over an MCP server and selects the IP with maximum keyword overlap against the planner’s search query. Subblocks tagged GENERATE are passed to a separate *codegen* call. The *integrator* concatenates the corpus IP source files with the generated logic and asks an LLM to write the TopModule wiring. The result is compiled by iverilog -g2012 and verified against the ChipBench testbench.

B. What we score

The benchmark in this paper isolates the *routing* stage. We hold the corpus, the router, and the gold labels fixed across all 15 LLMs. The only thing that changes is the planner, so any difference in the score is a property of the planner LLM. For every problem we have hand-labeled gold consisting of (a) a set of corpus IP IDs that a sensible planner-plus-router should surface, and (b) a per-role decision of whether each subblock should be REUSE_IP or GENERATE.

We track three metrics. Precision is the fraction of router-picked IPs that are also in the gold IP set. Recall is the fraction of gold IPs the router picked. Kind agreement is the per-subblock fraction where the planner’s REUSE_IP/GENERATE decision matches the gold label, computed via best-effort name matching between planner subblock names and gold role labels. F1 is our headline number because a model that picks one correct IP and ignores everything else scores a misleadingly high precision.

C. The corpus and gold labels

The corpus is 30 IP modules across six categories: memory (sync/async FIFO, single/dual-port RAM, register file), datapath (32-bit ALU, 16x16 multiplier, 8-bit divider, ripple/CLA adder, subtractor, comparator, barrel shifter), counter (up, down, up/down, BCD, gray-code, Johnson, frequency divider), sequence (LFSR, edge detector, shift register, sign-extender), primitive (T flip-flop, 2/4/8-input mux, 3-to-8 decoder, priority encoder), and clock (frequency divider). Every IP ships with a parameterizable Verilog implementation and an Icarus self-test that we run as part of CI. The set was sized to cover the architectural shapes that appear in ChipBench’s hierarchical `not_self_contain` and `cpu_ip` sets, plus seven additions from the `self_contain` set (multiplexer, dual-port RAM, up/down counter, Johnson counter, gray counter, fractional frequency divider, pipelined multiplier). All IPs ship under MIT.

The gold labels live in `eval/gold_labels.py` as a Python dict, written by hand by one of us after reading each problem’s specification. Each entry records the expected corpus IP IDs, the per-role kind decisions, and a short rationale. The labels reference IP IDs from the full 30-IP corpus, so labels written before a corpus expansion remain stable.

III. IMPLEMENTATION

A. Models tested

We ran 15 LLMs across four providers. From Anthropic via the `claude` CLI: Opus 4.7, Sonnet 4.6, Haiku 4.5. From OpenAI via the `chat-completions` API: GPT-5.4, 5.2, 5.1, 5, and 4.1. From AWS Bedrock in `us-east-1` via the `converse` API: DeepSeek-R1 (through the `us.deepseek.r1-v1:0` inference profile) and DeepSeek V3.2. From Google Gemini via the `generativelanguage.googleapis.com/v1beta` REST endpoint: 2.5 Pro, 2.5 Flash, 2.5 Flash-Lite, 3 Flash Preview, and 3.1 Flash-Lite Preview. Three additional Gemini variants (3 Pro Preview, 3.1 Pro Preview) returned 503s

TABLE I
HEADLINE ROUTING SCORES, BEST-OF-PROVIDER, 16-PROBLEM EVAL,
 $n = 16$ PER CELL, SINGLE SEED.

Provider	Model	Precision	Recall	F1	Kind agree.
OpenAI	GPT-5.2	0.510	0.463	0.467	0.753
Bedrock	DeepSeek V3.2	0.458	0.429	0.441	0.777
Google	Gemini 2.5 Flash-Lite	0.500	0.417	0.437	0.719
Anthropic	Claude Opus 4.7	0.396	0.365	0.375	0.901

on the majority of calls during our run window and were excluded.

The protocol is identical across all 15 models. Same planner system prompt, no few-shot examples, temperature zero where supported, the same router, the same 30-IP corpus, and the same 16 problems. We use 16-way thread-pool concurrency with retries handled by the underlying SDKs.

For the headline numbers in Fig. 1(b) and Table I we report the four best-of-provider models with full $n = 16$ coverage: Claude Opus 4.7, GPT-5.2, DeepSeek V3.2, and Gemini 2.5 Flash-Lite.

B. Eval scaffolding

The full multi-LLM routing eval is one Python file (`eval/multi_routing.py`, ~160 lines). It loads the gold labels, fans out 240 calls across providers, records planner output and router decisions per call, and aggregates into per-(LLM, problem) records and a per-LLM summary. Per-call latency, raw planner output, parsed subblocks, and router decisions are all preserved in JSON for post-hoc analysis. The figure pipeline (`eval/make_figures.py`) regenerates ten plots from the JSON without re-running the eval.

C. Cost and time

A single full sweep across all 15 models ran in under six minutes wall-clock at 16-way concurrency. The most expensive provider per call was OpenAI (~\$0.03 for GPT-5.4 on a typical planner call, dominated by output tokens). DeepSeek-R1 on Bedrock was the cheapest at roughly \$0.30 for the entire 9-problem run including reasoning-trace tokens. The full reproducible eval, including all the nice-to-have per-provider sweeps we did during development, came in under \$25 total.

IV. RESULTS

A. Routing quality is its own axis

Our central finding is in Fig. 1(b). The four colored dots are the best-of-provider headline models. The grey dots are the other 11 LLMs. The x-axis is scratch pass rate on the `cpu_ip` set — single-shot Verilog through the ChipBench testbench. The y-axis is routing F1 against our hand-labeled gold on the same 16 problems. If routing skill were a side-effect of overall model strength, the dots would line up along the diagonal. They don’t.

The F1 spread across all 15 models is 0.39 (0.13–0.52); the pass-rate spread on `cpu_ip` is 0.11 (most models pass

exactly 1/9). On a single-axis benchmark the models would be indistinguishable; on rtl-mosaic they separate cleanly.

B. Provider family does not predict the winner

Table I also undercuts the assumption that the strongest provider wins. DeepSeek V3.2 (Bedrock) ties with Anthropic Claude on kind agreement and beats it on F1 by 0.066. Within OpenAI, GPT-5.2 outperforms the newer GPT-5.4 in our run. We do not claim GPT-4.1 is a smarter model than GPT-5.4 in general — we claim that holding the planner prompt constant across all models, which is the only way to get apples-to-apples comparisons, surfaces the prompt-sensitivity of each model in different ways. A single planner prompt is not a faithful read on a model’s full capability, but it is the right protocol for a first-cut benchmark.

C. Failure modes

We observed four recurring failure modes across the 240 calls. (1) *Hallucinated picks*: the planner correctly tags a 32-bit ALU as REUSE_IP but emits a search query the keyword router resolves to `priority_encoder`. The router does string overlap; bad query in, bad pick out. (2) *Over-decomposition*: weaker models split a five-line spec into six GENERATE subblocks and never reuse anything. (3) *Under-decomposition*: GPT-5 returns one big GENERATE block on average (1.0 subblocks per problem), so the router never gets called. This is why GPT-5 sits at the bottom of our leaderboard despite being a competent code model. (4) *Kind-flip*: at temperature zero, some Anthropic models swing between REUSE_IP and GENERATE on the same subblock across reruns. Claude Opus 4.7 went from F1=0.678 in an early 9-problem run to F1=0.375 in our final 16-problem run. Run-to-run variance is real even at deterministic settings; multi-seed averaging is the obvious follow-up we did not yet do.

D. The corpus-disambiguation bottleneck

When we expanded the corpus from 20 to 30 IPs, mean F1 across the eval dropped, not rose. The reason is corpus disambiguation. Adding `up_down_counter`, `johnson_counter`, and `gray_counter` alongside the existing `up_counter` created near-duplicate keyword profiles. A planner emitting `search_query="counter"` now resolves to whichever IP keyword matches first, not the right one. Replacing the keyword router with embedding similarity would directly address this, and we believe it would lift the mean F1 by 0.05–0.10. We list this as the highest-priority follow-up.

V. RELATED WORK

ChipGPT [5], VeriGen [6], RTLCoder, and HiVeGen target scratch Verilog generation; HiVeGen retrieves snippets from code corpora rather than IPs. ChipNeMo [7] domain-adapts an open model on EDA scripts. MAGE [8] multi-agent decomposes per-subtask scratch generation. SiliconMind-V1 [2] multi-agent generates one module at a time and is the substrate we extend. None of these works score reuse against an IP catalog.

The benchmark side is similar. VerilogEval [4] is single-module; reuse is degenerate. RealBench targets full SoCs but does not separately score IP retrieval. ChipBench [3] ships an `cpu_ip` subset that is hierarchical in structure but is scored on testbench pass rate alone. Our contribution is the reuse-aware metric, the gold labels, and the multi-LLM eval protocol — not a new training pipeline.

VI. CONCLUSION

A reuse-aware metric separates LLMs that scratch-only benchmarks group together. The planner is the bottleneck of the system we built: holding the router and corpus fixed across 15 models, an F1 spread of 0.39 lives entirely in subblock decomposition and search-query specificity. Three follow-ups stand out. First, replace the keyword-overlap router with embedding similarity over IP descriptions to fix the corpus-disambiguation bottleneck. Second, multi-seed averaging to put error bars on every cell of the leaderboard. Third, fine-tune a small open model on planner traces from the strongest LLM, in the SiliconMind style. Code, corpus, gold labels, and run scripts are open at <https://github.com/MattKotzbauer/rtl-mosaic>.

Self-assessment. We met the project goal of producing a working harness and a defensible benchmark, but we did not show the harness end-to-end beating scratch generation on the `cpu_ip` set — both are stuck at 1/9. The routing-quality finding is what we stand behind. Given another semester, we would build the test-feedback retry loop, run multi-seed for variance, and push the eval out as a public leaderboard.

TEAM CONTRIBUTIONS

Matt Kotzbauer: 30-IP corpus and self-tests, multi-provider runner, gold labels, figure pipeline, presentation deck, this report. **Warren Zhu**: planner, integrator, MCP IP-router, end-to-end harness CLI. **Leonardo Ferreira**: ChipBench baseline runner, eval metrics module, slide deck logistics, debugging on the iverilog side.

REFERENCES

- [1] Semico Research, “Semiconductor IP market: SoC share by area,” Industry report, 2024.
- [2] SiliconMind-V1: “Multi-agent distillation and debug for Verilog generation,” *arXiv preprint arXiv:2603.08719*, 2026.
- [3] ChipBench: “A realistic-module benchmark for LLM-driven RTL design,” *arXiv preprint arXiv:2601.21448*, 2026.
- [4] M. Liu et al., “VerilogEval: Evaluating large language models for Verilog code generation,” NVIDIA, *arXiv arXiv:2309.07544*, 2023.
- [5] K. Chang et al., “ChipGPT: How far are we from natural language hardware design,” *arXiv arXiv:2305.14019*, 2023.
- [6] S. Thakur et al., “VeriGen: A large language model for Verilog code generation,” *arXiv arXiv:2308.00708*, 2023.
- [7] M. Liu et al., “ChipNeMo: Domain-adapted LLMs for chip design,” NVIDIA, *arXiv arXiv:2311.00176*, 2023.
- [8] “MAGE: A multi-agent engine for automated RTL design,” *arXiv arXiv:2412.07822*, 2024.